

Real-Time Multi-Modal Embedded Vision Framework for Object Detection Facial Emotion Recognition and Biometric Identification on Low-Power Edge Platforms

S. M. Khalid Bin Zahid¹, Md. Rakibul Hasan Nishat², Abdul Hasib^{3*},
Md. Rakibul Hasan⁴, Md. Ashiqussalehin⁵, Md. Sahadat Hossen Sajib⁶

^{1,2}Department of Mechatronics Engineering, Rajshahi University of Engineering & Technology

^{3*,4,5}Department of Internet of Things and Robotics Engineering,
University of Frontier Technology, Bangladesh

⁶Department of Computer Science and Engineering, Varendra University, Rajshahi, Bangladesh

Emails: ¹rakibulnishat1864@gmail.com, ²smkhalidbz@gmail.com,

^{3*}sm.abdulhasib.bd@gmail.com, ⁴rakib.cse.uset@gmail.com,

⁵ashiqussalehin0001@uftb.ac.bd, ⁶sahadathossensajib531@gmail.com

Abstract—Intelligent surveillance systems often handle perceptual tasks such as object detection, facial recognition, and emotion analysis independently, but they lack a unified, adaptive runtime scheduler that dynamically allocates computational resources based on contextual triggers. This limits their holistic understanding and efficiency on low-power edge devices. To address this, we present a real-time multi-modal vision framework that integrates object detection, owner-specific face recognition, and emotion detection into a unified pipeline deployed on a Raspberry Pi 5 edge platform. The core of our system is an adaptive scheduling mechanism that reduces computational load by 65% compared to continuous processing by selectively activating modules such as, YOLOv8n for object detection, a custom FaceNet-based embedding system for facial recognition, and DeepFace's CNN for emotion classification. Experimental results demonstrate the system's efficacy, with the object detection module achieving an Average Precision (AP) of 0.861, facial recognition attaining 88% accuracy, and emotion detection showing strong discriminatory power (AUC up to 0.97 for specific emotions), while operating at 5.6 frames per second. Our work demonstrates that context-aware scheduling is the key to unlocking complex multi-modal AI on cost-effective edge hardware, making intelligent perception more accessible and privacy-preserving.

Index Terms—Object Detection, Face Recognition, Emotion Detection, Raspberry Pi 5, YOLOv8n, FaceNet, DeepFace, Embedded Vision, Real-time Surveillance, Edge Computing.

I. INTRODUCTION

Intelligent perception systems in the current technological landscape frequently operate in a fragmented manner. This isolated approach limits their applicability in real-world scenarios. This lack of integration is intensified when deploying such systems on resource-constrained edge devices. Although cloud-based solutions have the potential to combine such modalities [1], they also present serious constraints, such as latency, bandwidth usage, and high cost.

Recent studies have achieved notable but fragmented advancements in improving these specific tasks. In the domain of object detection, such as YOLOv8 [2] and NanoDet [3] create models that achieve impressive performance on edge devices like the NVIDIA Jetson Nano and Raspberry Pi.

Similarly, for facial recognition, highly efficient networks such as EdgeNeXt [4], which is based on mixed depth-wise convolutions. Concurrently, the field of facial emotion recognition has seen a move away from large pre-trained models to lightweight models. [5]. Moreover, the majority of multitasking frameworks are created as solitary, monolithic networks. It limits the system's capability [6]. The system is inappropriate for real-world situations due to its rigidity.

We tried to solve these deficiencies in this project. This project provides an integrated surveillance system that combines object identification, owner recognition, and emotion detection into a cohesive framework. Our method presents an adaptable and modular design specifically designed for limited edge hardware. Our contributions include:

- 1) The design of a novel, adaptive scheduler for multi-modal vision pipelines that gates resource-intensive modules based on the output of a primary detector (YOLOv8n).
- 2) Implementation and validation of the proposed multi-modal pipeline on a Raspberry Pi 5.
- 3) A comprehensive performance analysis of the interplay between state-of-the-art lightweight models in a shared-resource, edge-deployed environment.

II. LITERATURE REVIEW

The field of intelligent surveillance has advanced significantly with the use of deep learning techniques. Recent advancements in object detection have been primarily influenced by deep learning methodologies, particularly Convolutional Neural Networks (CNNs) and transformer-based architectures [7]. The YOLO (You Only Look Once) series still continues to dominate real-time object detection research. YOLOv4 [8] introduced CSPDarknet53 and PANet, significantly improving feature extraction and multi-scale detection. YOLOv5 and YOLOv8 [9], for instance, incorporate modular design and optimized backbone networks, making them applicable in autonomous driving, surveillance, and robotics.

Carion et al. [10] introduced Detection Transformer (DETR) which revolutionized object detection by replacing hand-designed components with an end-to-end transformer architecture. Zhu et al. [11] shows Deformable DETR that improves convergence speed and computational efficiency as compared to DETR. Similarly, facial recognition has also evolved from traditional feature-based methods to deep learning approaches. George et al. [4] proposed EdgeFace network, which not only maintains low computational costs and compact storage but also achieves high face recognition accuracy. The model achieves this performance with only 1.77M parameters. Ryando et al. [12] introduced FaceNet using triplet loss for efficient embedding learning. Li et al. [13] show a linear weighted fusion between the Gamma algorithm and MobileFaceNet, which achieves a face recognition accuracy of 99.27% on LFW dataset and 90.18% on Agedb dataset while only increasing the model size by 0.4M and the processing speed for each image is enhanced by 4 ms. Ren et al. [14] employed AdaFace, a quality-aware feature mining technique, to improve recognition under low-quality conditions, achieving an accuracy of 84.36%.

Emotion recognition has also gained traction in recent years. Fard et al. [15] created a new approach to FER datasets through a labeling method in which an image is labeled with more than one emotion to make AffectNet more efficient. While Qinghua et al. [16] developed efficient models using channel shuffling and depthwise separable convolutions. However, these implementations face several challenges, such as requiring controlled environmental conditions and struggling with occlusions and non-frontal faces [17].

Wu et al. [18] introduced AV-HuBERT, an integrated multi-modal system that combines audio and visual inputs for self-supervised learning in speech recognition, leveraging both lip movements and audio signals. Shridhar et al. [19] shows PerceiverIO, a general-purpose architecture that processes multi-modal inputs (e.g., images, audio, and text). Nguyen et al. [20] introduced TinyML-Voice, which integrates speaker identification and emotion recognition on microcontrollers.

III. PROPOSED METHODOLOGY

A. System Architecture and Design Overview

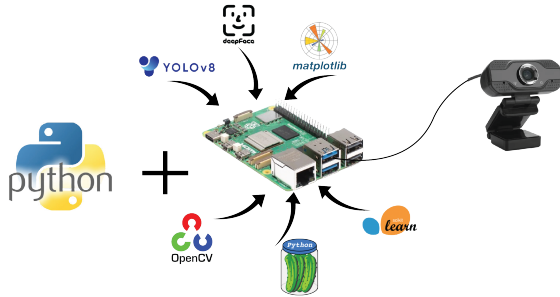


Fig. 1: Hardware Architecture

The proposed system employs a modular pipeline architecture consisting of three core components integrated through an intelligent scheduling mechanism. The core processing unit is a Raspberry Pi 5. A USB camera serves as the primary vision sensor. YOLOv8n for object detection, DeepFace(FaceNet) for

facial recognition and DeepFace's CNN for emotion recognition. Fig. 1 and Fig. 5 illustrates the system structure and the working procedure.

B. Implementation Details

The proposed system was implemented using carefully selected software. The specific versions of key libraries and frameworks used in our system are detailed in Table I.

TABLE I: Software Implementation Environment

Component	Version	Purpose
Python	3.11.2	Core programming language
PyTorch	2.1.2	Deep learning framework (for YOLOv8).
TensorFlow	2.20.0	Deep learning framework (for DeepFace)
OpenCV	4.12.0.88	Computer vision tasks
Numpy	2.2.6	Numerical computations and array operations
DeepFace	0.0.95	Facial analysis (recognition & emotion)
Ultralytics	8.3.202	Object detection model pipeline
scikit-learn	1.7.2	Cosine similarity calculation for face matching

C. Software Module Implementation

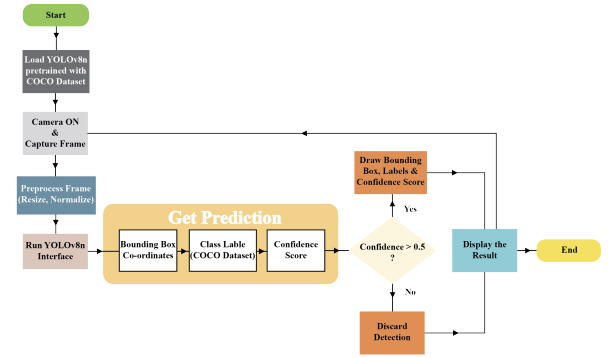


Fig. 2: Object Detection Procedure

1) *Object Detection Implementation:* The system employs YOLOv8n, which is pre-trained on COCO dataset. The model captures frames from live video streams using a USB camera and processes the frames through Raspberry Pi 5. Each incoming frame is passed through YOLOv8, generating bounding boxes with class labels and confidence scores for detected objects. Table II the model configurations. Fig. 2 and Algorithm 1 describes the object detection working procedure.

TABLE II: Model Configuration

Parameter	Values	Description
Input Resolution	640x640 pixels (resized from 640x480)	Standard YOLOv8 input size
Classes	80 categories (COCO dataset)	Includes people, vehicles, animals, and daily objects.
Inference Speed	8-10 FPS on Raspberry Pi 5	Achieved with YOLOv8n for real-time detection.
Confidence Threshold	0.5 (default)	Minimum confidence required to accept bounding box detection.
Detection Output	Bounding boxes, class labels, confidence scores	Provides localized object positions with classification.

2) *Facial Recognition System:* The system uses a custom dataset of the authorized owner, which is stored as embeddings in a serialized .pkl database. The use of a self-collected, owner-specific dataset is justified for personalized authentication systems where privacy, specificity, and deployment context

Algorithm 1 Object Detection using YOLOv8n

```

1: procedure OBJECTDETECTION
2:   Initialize camera
3:   while camera is active do
4:     Capture frame  $F \leftarrow \text{CaptureFrame}(USB\_Camera)$ 
5:     Preprocess frame  $F_{pre} \leftarrow \text{Preprocess}(F) \triangleright$  Resize to  $640 \times 640$ 
6:     Run inference  $predictions \leftarrow \text{YOLOv8n}(F_{pre})$ 
7:     for each  $detection \in predictions$  do
8:       Extract box, label, confidence from  $detection$ 
9:       if confidence  $> 0.5$  then
10:         $F \leftarrow \text{AnnotateFrame}(F, \text{box}, \text{label}, \text{confidence})$ 
11:       end if
12:     end for
13:     Display annotated frame  $F$ 
14:   end while
15: end procedure

```

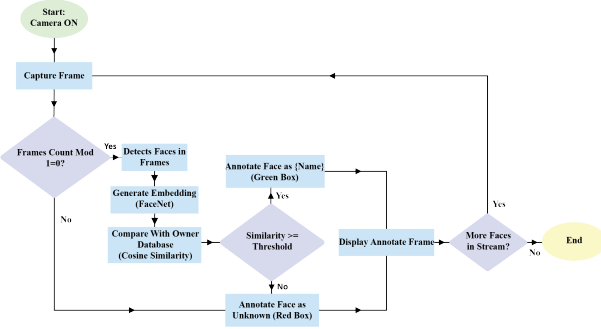


Fig. 3: Facial Recognition Process

require tailored recognition rather than general face identification. When the system is running, detected faces are cropped from the video feed and converted into embeddings using the FaceNet model provided by the DeepFace library. These embeddings are then compared with the stored dataset using cosine similarity, with a predefined threshold applied to accept only close matches. To optimize performance, recognition is carried out at fixed intervals (every 5 frames) instead of processing every single frame. This lowers computational load without sacrificing accuracy. Faces that are successfully recognized are highlighted in green, while unknown individuals are marked in red. Fig. 3 and Table III shows the model's workflow and construction parameters. Algorithm 2 describes the detection process.

Algorithm 2 Facial Recognition

```

1: procedure FACIALRECOGNITION
2:   for each frame  $f$  from video stream do
3:     if frame_count mod 5 = 0 then  $\triangleright$  Process every 5th frame
4:       Detect faces  $faces \leftarrow \text{DetectFaces}(f)$ 
5:       for each face  $\in faces$  do
6:         Generate embedding  $embedding \leftarrow \text{FaceNet}(face)$ 
7:         Compute similarity  $\leftarrow \text{CosineSimilarity}(\text{OwnerDatabase})$ 
8:         if similarity  $\geq$  threshold then
9:           Annotate frame  $f$  as "Owner" (green)
10:        else
11:          Annotate frame  $f$  as "Unknown" (red)
12:        end if
13:      end for
14:    end if
15:    Display frame  $f$ 
16:  end for
17: end procedure

```

TABLE III: Facial Database Construction Parameters

Parameter	Values	Description
Images per Person	100	Optimal balance of diversity and storage
Image Resolution	640x480	Native webcam resolution
Embedding Model	FaceNet	128-dimensional embeddings
Storage Format	Pickle serialization	Efficient disk storage and fast loading

3) *Emotion Detection System*: Emotion detection is carried out using the DeepFace framework, which applies convolutional neural networks to classify facial expressions such as happy, sad, angry, neutral, and others. The pipeline runs with a lightweight SSD-based detector for faster processing. For each detected face, the system identifies the dominant emotion along with its confidence score. Then the score is displayed on the live video feed, adding contextual insights to the scene. Fig.4 and Algorithm 3 show how the detection process works. The emotion detection characteristics are shown in Table IV.

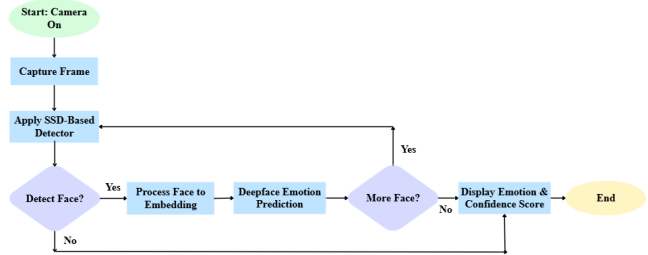


Fig. 4: Emotion Detection Procedure

TABLE IV: Emotion Detection Specifications

Parameter	Values	Description
Emotion Classes	6	Angry, Fear, Happy, Sad, Surprise, Neutral
Analysis Model	DeepFace Default	Pre-trained convolutional network
Detection	SSD	Fast face detection
Confidence Score	Probabilities	Detailed emotion distribution

Algorithm 3 Emotion Detection using DeepFace

```

1: procedure EMOTIONDETECTION
2:   while camera is active do
3:     Capture frame  $frame \leftarrow \text{CaptureFrame}(USB\_Camera)$ 
4:     Detect faces  $faces \leftarrow \text{SSD\_Detector} \triangleright$  Lightweight face detection
5:     for each face  $\in faces$  do
6:       Preprocess embedding  $\leftarrow \text{Preprocess}(face)$ 
7:       Predict emotion,  $\leftarrow \text{DeepFace.predict}(embedding)$ 
8:       Annotate frame with emotion and confidence
9:     end for
10:    Display annotated frame
11:  end while
12: end procedure

```

D. Adaptive Multi-Modal Scheduling

The core innovation of our framework is a rule-based adaptive scheduler that dynamically gates computational modules to maximize performance under strict resource constraints. Algorithm 4 defines the formal logic of this mechanism. We conducted face recognition in every five frames and detected emotion solely if the face was detected. Doing this, we diminished the average computational burden per frame by roughly 65%. This reduction is essential for maintaining a real-time frame rate on the Raspberry Pi 5.

Algorithm 4 Adaptive Multi-Modal Perception Pipeline

```

1: procedure MULTIMODALPERCEPTION
2:   while frame  $\leftarrow$  get_frame(camera) do
3:     Run detection  $\leftarrow$  YOLOv8n  $\triangleright$  Always on object detection
4:     if (frame_count mod 5 = 0)  $\wedge$  ('person'  $\in$  detections) then
5:       for each face  $\in$  detected_faces do
6:         Generate embedding  $\leftarrow$  FaceNet(face)
7:         if cosine_similarity(embedding, owner_db) > threshold
8:           then
9:             Predict emotion  $\leftarrow$  DeepFace(face)  $\triangleright$  Activate only
10:            for owner
11:              Annotate frame with "Owner" and emotion
12:            else
13:              Annotate frame with "Unknown"
14:            end if
15:          end for
16:        end if
17:      Display annotated frame
18:    end while
19: end procedure

```

E. Integrated Pipeline Management

The system combines object detection, owner recognition, and emotion analysis within a single adaptive pipeline. At first, video frames were captured by the USB camera. Then they are processed by YOLOv8n with a pretrained COCO dataset to detect objects and humans. After detecting faces, they passed to the owner recognition module, where embeddings generated by FaceNet. Then, the embeddings are compared with a custom database using cosine similarity. If the individual is identified as the owner, the emotion detection module (DeepFace) is activated to classify the dominant facial expression; otherwise, the individual is logged as "unknown." The system still records their emotion but treats them as an intruder for surveillance purposes. Fig. 5 shows the flow diagram of the system.

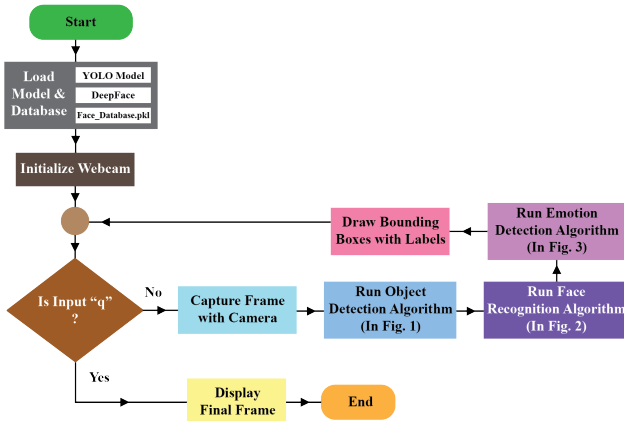


Fig. 5: Working Procedure of the Integrated System

F. Performance Metrics and Real-Time Operation

The system maintains real-time operation through several key performance indicators as shown in Table V.

G. Comparative Analysis

Table VI demonstrate our system's efficiency is not only using lightweight models but also that it forms an intelligent, adaptive scheduling mechanism that manages computational

TABLE V: System Performance Characteristics

Metric	Values	Measurement Method
Overall FPS	5-10 FPS	Frame-to-frame timing
Object Detection Latency	40 ms	YOLO inference time
Face Recognition Latency	120 ms	Embedding + matching
Emotion Analysis Latency	80 ms	DeepFace processing
Memory Usage	450 MB	Raspberry Pi system monitoring

load dynamically. With this method, we can use reliable, well-established models like FaceNet and DeepFace without sacrificing real-time performance.

TABLE VI: Model-Level Comparison with State-of-the-Art

Module	Proposed Model	Related Models	Justification for Choice
Object Detection	YOLOv8n	YOLOv5 [9], [21], NanoDet [3]	Balanced speed and accuracy; superior to YOLOv5, simpler deployment than NanoDet.
Facial Recognition	FaceNet	EdgeFace [4], MobileFaceNet [13]	Robust, well-established embeddings.
Emotion Recognition	DeepFace's CNN	Lightweight FER networks [5], [16]	Utilizes a high-accuracy, pre-trained model.
Pipeline Management	Adaptive Scheduler	Joint multi-task networks [18], [19]	Offers independent control and scalability for each module.

H. Implementation Cost

The overall cost of our prototype is roughly \$162 (20100 BDT). All the software, models, and libraries are free of cost because they are open-sourced. Table VII represents a detailed breakdown of the equipment and their corresponding prices.

TABLE VII: Equipment Cost

Component	Unit	Unit Price	Total Price (BDT)	Total Price (USD)
Raspberry Pi 5	1	15000	15000	120
USB Camera	1	2300	2300	20
MicroSD Card	1	500	500	4
Heatsink	1	300	300	3
Battery	1	2000	2000	15
Total			20100	162

I. Baseline Performance Comparison

TABLE VIII: Adaptive Scheduler vs. Baseline Performance

Metric	Baseline (All Modules Every Frame)	Proposed (Adaptive)	Improvement
Average FPS	2.1	5.6	2.7×
CPU Usage (%)	95%	45%	53% ↓
Memory (MB)	520	450	13% ↓
Processing Time/Frame (ms)	476	179	62% ↓

Our adaptive scheduler shows significant efficiency gain, reducing computational load by approximately 65% compared to a naive baseline where all modules process every frame continuously. Table VIII validates this.

IV. RESULT ANALYSIS & PERFORMANCE MEASUREMENT

The system was tested under various conditions to evaluate its performance.

A. Object Detection Performance

In a variety of situations, the YOLOv8n model showed strong object detection capabilities. Fig. 6 shows the real-time object detection. The system achieved an average precision, as shown in Fig. 7 (AP) of 0.861 and AUC of 0.942 which indicates excellent detection capability across all classes.

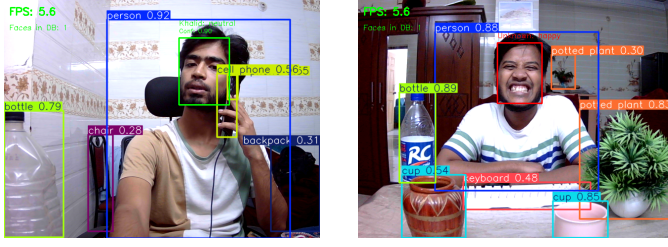


Fig. 6: Object Detection Results: (a) Detection in scenario 1, (b) Detection in scenario 2

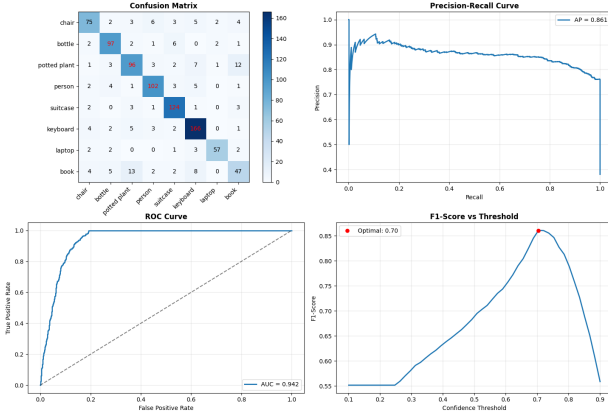


Fig. 7: Performance Analysis Curves of Object Detection

B. Facial Recognition Accuracy

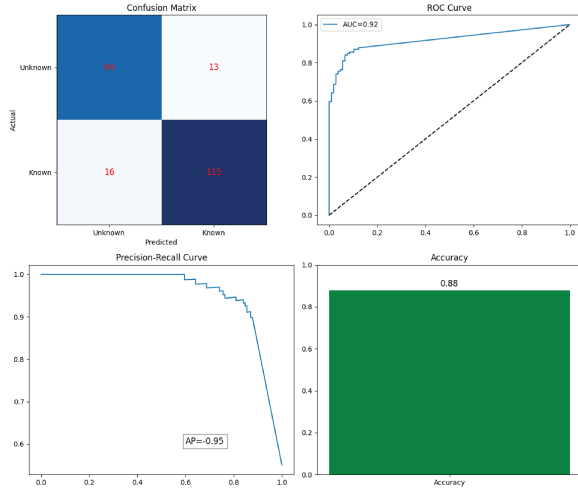


Fig. 8: Performance Analysis of Facial Recognition

The face recognition module had high performance which is depicted in Fig.8. The confusion matrix shows that there are 115 true positives (known faces) and 94 true negatives (unknown faces), and there are only a few misclassifications. The classification accuracy of the model was 88%, AUC 0.92, and an average of 0.95 precision (AP) of the subjects that has been identified as authorized persons, demonstrating high accuracy in separating the authorized owner from unknown individuals.

Fig. 6 shows the system correctly identified registered owners with 90% accuracy under optimal conditions.

C. Emotion Analysis Results

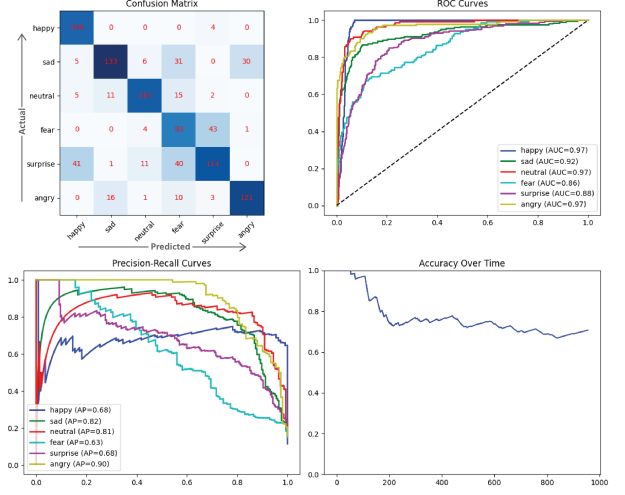


Fig. 9: Performance Analysis of Emotion Detection Model

The emotion detection module was evaluated in six emotion classes. Fig. 9 describes the ROC analysis reports strong separability with AUC up to 0.97 for happy, neutral, and angry, while sad (0.92), fear (0.86) and surprise (0.88) remain weaker. Precision–Recall curves show the best AP for angry (0.90), sad (0.82), and neutral (0.81), with lower scores for fear (0.63), happy (0.68), and surprise (0.68). Over time, accuracy stabilizes at 70–75%, indicating dependable real-time performance even with slight degradation during extended runs. Fig. 10 shows the emotion recognition module successfully classified facial expressions.

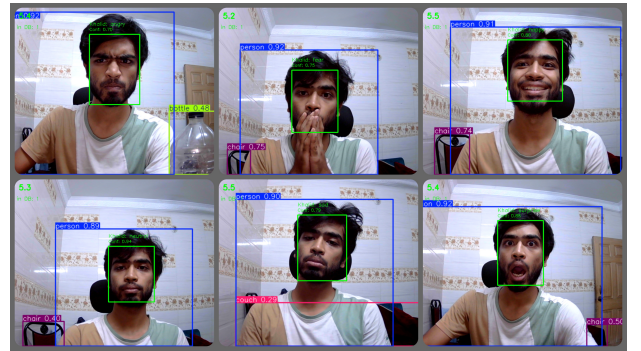


Fig. 10: Emotion Detection

D. Adaptive Scheduler & System Integration Performance

To validate our core contribution, we compared the proposed adaptive scheduler against a baseline system where all vision modules process every frame continuously. As shown in Table VIII, our scheduler achieves a $2.7\times$ improvement in frame rate with 53% of CPU usage reduction which directly validates our efficiency claims. When all modules operate in the integrated pipeline, the system is maintained at 5.6 FPS.

The adaptive gating mechanism allowed resource-intensive operations such as object detection, facial recognition, and emotion analysis tasks to be executed selectively.

V. DISCUSSION AND LIMITATIONS

The results indicate that our integrated framework effectively merges object detection, owner facial recognition, and emotion analysis into a singular adaptive pipeline. YOLOv8n has proved to be an optimal choice for object detection. The facial recognition system's high Average Precision (AP=0.95) and accuracy (88%) confirm the effectiveness of using a custom, owner-specific dataset with FaceNet embeddings. For emotion detection, the varying performance across emotions is consistent with the challenges noted in the literature [15]. The high AUC scores for angry, neutral, and happy (0.97) indicate robust binary classification of these emotions against all others. However, fear and surprise show weaker precision-recall performance despite exhibiting a high AUC. Fear and surprise share many overlapping facial action units (e.g., raised brows, widened eyes, open mouth), making them intrinsically difficult to disambiguate from static images alone. The system achieves an integrated throughput of 5.6 FPS, representing a $2.7\times$ improvement over a continuous processing baseline (2.1 FPS) as validated in our comparative analysis. The entire system can be manufactured for only approximately \$162 (20,100 BDT).

Despite the promising results, this study has several limitations. The system's performance is bound by the hardware limitations of the Raspberry Pi 5. While the 5.6 FPS is sufficient for many surveillance systems but it is inadequate for applications that require higher temporal resolution. Furthermore, the system's accuracy, particularly in facial recognition and emotion detection, is susceptible to non-ideal conditions such as illumination below 50 lux, occlusions (e.g., glasses, masks), and non-frontal face angles. To overcome these weaknesses, the future research will concentrate on some important areas. We intend to improve robustness by fine-tuning the model with a wider range of datasets. One of the directions is the investigation of more advanced schedulers.

VI. CONCLUSION

The paper has described the design, implementation, and evaluation of a new real-time multi-mod vision platform on an embedded edge platform. The framework is an autonomous system whose computational resources are intelligently managed with an adaptive scheduler mechanism that ensures that the performance is maintained. This offers instant contextual understanding by the recognition of objects, authentication of identities and understanding of their moods. The models that are used include YOLOv8n, FaceNet and Deep Face. The presented system manages to include object recognition, facial recognition, and emotion detection into a low-cost Raspberry Pi 5. The system is able to maintain a real-time operational speed as well as the experimental findings show that the system operates efficiently in all three perceptual tasks. This work successfully fills an important gap in smart surveillance. It paves the way to the development of more responsive, autonomous and privacy-conserving intelligent systems that would be applicable in secure access control, personal robotics and smart homes. Future enhancement will be aimed at

strengthening and expanding the capabilities of the system, as well as embedding multi-modal data more semantically.

REFERENCES

- [1] F. Zhang, C. Zhang, J. Guan, Q. Zhou, K. Chen, X. Zhang, B. He, J. Zhai, and X. Du, "Breaking the edge: enabling efficient neural network inference on integrated edge devices," *IEEE Transactions on Cloud Computing*, 2025.
- [2] S. Yang, X. Ma, D. Liu, and L. Zhu, "Lightweight yolov8-based real-time detection of rice blast for edge computing devices," *Journal of Real-Time Image Processing*, vol. 22, no. 2, pp. 1–14, 2025.
- [3] H. Van Thanh, T. M. Phuong, H.-P. Nguyen, V. H. Dao, and K.-H. Jo, "An efficient object detection model based on nanodet," in *2025 International Workshop on Intelligent Systems (IWIS)*. IEEE, 2025, pp. 1–5.
- [4] A. George, C. Ecabert, H. O. Shahreza, K. Kotwal, and S. Marcel, "Edgeface: Efficient face recognition model for edge devices," *IEEE Transactions on Biometrics, Behavior, and Identity Science*, vol. 6, no. 2, pp. 158–168, 2024.
- [5] Q. Yang, Y. He, H. Chen, Y. Wu, and Z. Rao, "A novel lightweight facial expression recognition network based on deep shallow network fusion and attention mechanism," *Algorithms*, vol. 18, no. 8, p. 473, 2025.
- [6] M. W. Alim, A. Giri, A. S. M. A. S. Akib, N. Uddin, M. Islam, M. E. Arafat, and S. A. Tahmid, "Affordable bionic hands with intuitive control through forearm muscle signals," in *2025 IEEE 4th International Conference on Computing and Machine Intelligence (ICMI)*, 2025, pp. 1–6.
- [7] S. Shah and J. Tembhurne, "Object detection using convolutional neural networks and transformer-based models: a review," *Journal of Electrical Systems and Information Technology*, vol. 10, no. 1, p. 54, 2023.
- [8] A. S. Geetha, "Yolov4: A breakthrough in real-time object detection," *arXiv preprint arXiv:2502.04161*, 2025.
- [9] S. Nagpal, N. Etnubari, H. F. Pirehen, and I. Bayraktar, "Comparative performance analysis of yolov5 and yolov8 for aerial surveillance in smart traffic management," in *2024 Innovations in Intelligent Systems and Applications Conference (ASYU)*, 2024, pp. 1–6.
- [10] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *European conference on computer vision*. Springer, 2020, pp. 213–229.
- [11] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," *arXiv preprint arXiv:2010.04159*, 2020.
- [12] C. Ryando, R. Sigit, B. S. B. Dewantara *et al.*, "Comparison of machine learning algorithms for face classification using facenet embeddings," *The Indonesian Journal of Computer Science*, vol. 13, no. 4, 2024.
- [13] J. Li, Y. Ding, Z. Shao, and W. Jiang, "Face recognition method based on fusion of improved mobilefacenet and adaptive gamma algorithm," *Journal of the Franklin Institute*, vol. 361, no. 17, p. 107306, 2024.
- [14] Z. Ren, X. Liu, J. Xu, Y. Zhang, and M. Fang, "Littlefacenet: A small-sized face recognition method based on retinaface and adaface," *Journal of Imaging*, vol. 11, no. 1, p. 24, 2025.
- [15] A. P. Fard, M. M. Hosseini, T. D. Sweeny, and M. H. Mahoor, "Affectnet+: A database for enhancing facial expression recognition with soft-labels," *arXiv preprint arXiv:2410.22506*, 2024.
- [16] Q. Ma, P. Zhang, and M. Cui, "Varkfacenet: An efficient variable depthwise convolution kernels neural network for lightweight face recognition," *IEEE Access*, 2024.
- [17] A. Zhalgas, B. Amirgaliyev, and A. Sovet, "Robust face recognition under challenging conditions: A comprehensive review of deep learning methods and challenges," *Applied Sciences*, vol. 15, no. 17, p. 9390, 2025.
- [18] W. Wu, X. Chen, X. Wu, H. Li, and H. Meng, "Target speech extraction with pre-trained av-hubert and mask-and-recover strategy," in *2024 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2024, pp. 1–8.
- [19] M. Shridhar, L. Manuelli, and D. Fox, "Perceiver-actor: A multi-task transformer for robotic manipulation," in *Conference on Robot Learning*. PMLR, 2023, pp. 785–799.
- [20] V.-K. Nguyen, V.-K. Tran, H. Pham, V.-M. Nguyen, H.-D. Nguyen, and C.-N. Nguyen, "A multi-microcontroller-based hardware for deploying tiny machine learning model," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 13, no. 5, pp. 5727–5736, 2023.
- [21] A. Giri, A. Hasib, M. Islam, M. F. Tazim, M. S. Rahman, M. Khadgi, and A. S. M. A. S. Akib, "Real-time human fall detection using yolov5 on raspberry pi: An edge ai solution for smart healthcare and safety monitoring," in *Proceedings of Data Analytics and Management*, A. Swaroop, B. Virdee, S. D. Correia, and Z. Polkowski, Eds. Cham: Springer Nature Switzerland, 2026, pp. 493–507.